SENTINEL PREVENTATIVE SERVICES

Credit Card Fraud Detection Platform

Complete Technical Design, ML Pipeline, Subprocess APIs, UI Features & Workspace Optimizations

Authors:	Machine Learning & Integration Engineering Team
Framework Stack:	React 19 + TanStack Start (TypeScript) + Scikit-Learn (Python)
Model Version:	Random Forest Classifier v4.2 (Standard Scaled)
Report Status:	Technical Specifications & Audited Review
Date:	July 3, 2026

1. Executive Summary

Sentinel is an intelligence platform engineered to detect credit card anomalies and transaction fraud in real-time. Historically, offline machine learning models remained disconnected from consumer endpoints. This project bridges that gap. By replacing mock analytics with a live Python pipeline running standard-scaled Random Forest inference, Sentinel allows fraud analysts to score individual card purchases and batch-validate bulk uploads through a beautiful dashboard.

2. Machine Learning Model & Pipeline

The model was trained on the European credit card dataset (284,807 transactions). Due to extreme class imbalance (only 492 fraud cases, or 0.172%), we employed a **StandardScaler** for variable normalization and a **Random Forest Classifier** trained with stratified cross-validation and class_weight='balanced' to prevent classification bias.

Model Metrics (Tested on 20% Stratified Split):

Metric	Trained Score	Target Evaluation Meaning
Accuracy	**99.91%**	Overall prediction accuracy rate
ROC-AUC	**97.90%**	General true positive / false positive separation
Recall	**82.65%**	82.65% of all active fraud successfully blocked
Precision	**71.68%**	High trust rating on flagged fraud; minimal false alerts
F1-Score	**76.78%**	Harmonic balance between precision and recall

Confusion Matrix Results:

	Predicted Genuine	Predicted Fraud
Actual Genuine	56,832 (True Neg)	32 (False Pos)
Actual Fraud	17 (False Neg)	81 (True Pos)

3. Full-Stack Web Architecture & Subprocess API

Sentinel's dashboard is built with React 19 and TanStack Start, compiling client routing and server endpoints together. To avoid the operational overhead of running persistent Python microservices (like Flask or FastAPI), the web server utilizes an on-demand child process pipeline. When a prediction is initiated, the server (src/server.ts) spawns a Python worker process, pipes the JSON request payload directly to stdin, and parses the stdout stream for final probabilities.

Asynchronous Child Process Code Segment (server.ts):

```
const py = spawn('python', [predictScriptPath], { cwd: projectRoot });
py.stdin.write(JSON.stringify(payload)); py.stdin.end(); py.stdout.on('data', (data) => { stdout += data.toString(); });
```

4. Real Input Metadata Binding & Risk Triggers

A major recent development was bridging user transaction metadata (Merchant, Channel, Location, Amount, and Card Network) with the model's abstract 28 PCA features (V1-V28). A deterministic hashing engine was introduced in the frontend (app.predictions.tsx) that maps text inputs to consistent, realistic PCA coordinates.

Dynamic Risk Triggers & Feature Skewing:

To let analysts test the ML model interactively, specific fraud risk rules were introduced that automatically skew V-features into known fraud distributions. In high-risk scenarios, features strongly negatively correlated with fraud (such as **V14**, **V17**, **V12**, and **V10**) are forced to extreme negative values, while positive correlation features (**V4** and **V11**) are shifted positive.

Determined Risk Conditions:

- ***High Amount Trigger****: Any transaction value greater than $1,200.*
- ***Online Gaming Purchase****: Merchant set to **Steam** and Amount greater than $150.*
- ***Electronics Purchase****: Merchant set to **Apple** and Amount greater than $800.*
- ***Suspicious Geolocation****: Location contains fields like 'unknown' or 'suspicious'.*

UI Integration (Auto-Derive Toggle):

An **'Auto-derive from metadata'** checkbox was added to the dashboard. When checked, the V1-V28 inputs are locked and derived automatically from the transaction details. When unchecked, inputs are unlocked, enabling analysts to override values and run custom simulations.

5. Workspace Optimization & File Cleanup

To make the project repository clean, optimized, and ready for deployment, we performed a destructive file audit. All redundant, large, and unused legacy assets were removed:

Removed Asset	Size	Reason for Removal
clarity-fraud-watch-main.zip	81.3 MB	Redundant backup archive of the active application folder.
ML part/ Directory	17.7 MB	Legacy academic workspace. Contained Jupyter notebooks for unrelated models (Decision Tree, KNN, SVM), LaTeX project source zips, and duplicate files.

This optimization saved **99.0 MB** of disk space and accelerated deployment pipeline caching.

6. System Verification & Test Cases

Verification was conducted using mock inputs. Changing parameters in the predictions portal yields consistent classifications:

Verification Test Case Results:

Scenario	Inputs	Derived Features	Model Prediction
1. Low-Risk Purchase	Amazon, In-store, SF Location, $25.00	V1-V28 close to zero (between -0.8 and 0.8)	**Genuine** Prob: 0.86%
2. High-Risk Gaming	Steam, E-commerce, NY Location, $250.00	V14: -12.0 V17: -10.0 V4: 6.0	**Fraudulent** Prob: 68.97%

7. Academic Publication Reference

To document this design formally, we compiled an IEEE format paper: Sentinel_IEEE_Paper.pdf. It details the full-stack architecture, Random Forest hyper-parameters, and standard scaling mathematics, providing a blueprint for deploying machine learning micro-kernels inside production web runtimes.

Thank you for using Sentinel Prevention Services.